

Relazione — Versione 3: Generic Parser

Obiettivo della Versione 3

La Versione 3 introduce il Generic Parser.

Nella Versione 2 il parser funzionava, ma era ancora rigido.

Il problema principale era che il parser ragionava con condizioni specifiche:

```
if parameters == ["name"]:  
    ...  
  
if parameters == ["a", "b"]:  
    ...
```

Questo approccio va bene con pochi tool, ma non scala.

Se in futuro aggiungiamo nuovi parametri o nuovi tool, il parser rischia di diventare pieno di `if`.

L'obiettivo della Versione 3 è rendere il parser più generico, flessibile ed estendibile.

Cosa cambia rispetto alla Versione 2

Nella Versione 2 il parser controllava direttamente le combinazioni di parametri.

Nella Versione 3 invece il parser segue questo ragionamento:

```
leggo il tool scelto  
↓  
guardo quali parametri richiede  
↓  
per ogni parametro cerco l'extractor corretto  
↓  
estraggo il valore  
↓  
costruisco arguments
```

Quindi il parser non ragiona più per tool specifico.

Ragiona per parametri.

Questa è la vera evoluzione della Versione 3.

Nuovo concetto: Parameter Extractors

La Versione 3 introduce gli extractor dei parametri.

Un extractor è una piccola funzione che sa come estrarre un certo valore.

Per esempio:

```
"name" → estrae il nome  
"a" → estrae il primo numero  
"b" → estrae il secondo numero
```

Questi extractor vengono collegati ai parametri tramite un dizionario:

```
parameter_extractors = {  
    "name": ...,  
    "a": ...,  
    "b": ...  
}
```

In questo modo il parser può lavorare in modo generico.

Parse Context

Un'altra idea importante della Versione 3 è il parse context.

Il context prepara in anticipo le informazioni utili dalla richiesta dell'utente.

Per esempio:

```
Add 2 and 3
```

può diventare:

```
{  
    "numbers": [2, 3],  
    "name": None  
}
```

Il parser poi usa questo context per estrarre i singoli parametri.

Questa scelta evita di ripetere più volte la stessa estrazione.

Flusso del Generic Parser

Esempio:

```
What is 2 + 3?
```

Il Router sceglie:

```
"addition"
```

Il tool `addition` richiede:

```
["a", "b"]
```

Il parser costruisce il context:

```
{  
  "numbers": [2, 3],  
  "name": None  
}
```

Poi estrae:

```
"a" → primo numero → 2  
"b" → secondo numero → 3
```

Il risultato finale del parser è:

```
{  
  "a": 2,  
  "b": 3  
}
```

Flusso completo della Versione 3

Il flusso dell'agente diventa:

```
User request  
↓  
Router  
↓
```

```
Generic Parser
↓
Executor
↓
Memory
```

I componenti principali sono ormai ben separati:

```
Router = sceglie il tool
Parser = estrae gli argomenti
Executor = valida ed esegue
Agent = coordina
Memory = salva lo stato
```

Questa è già una base molto più solida rispetto alla Versione 1.

Perché il Generic Parser è importante

Il Generic Parser è importante perché rende il sistema più estendibile.

Se in futuro aggiungiamo un nuovo tool, non vogliamo modificare il parser con tanti nuovi `if`.

Vogliamo solo dire:

```
questo parametro si estrae così
```

Per esempio, se un giorno aggiungiamo un tool meteo:

```
weather(city)
```

potremo aggiungere un extractor per `city`, invece di riscrivere tutto il parser.

Questa è una mentalità più professionale.

Cosa abbiamo imparato nella Versione 3

La Versione 3 insegna un concetto fondamentale:

il parser non deve conoscere ogni tool nel dettaglio. Deve sapere come estrarre parametri.

Questo rende il codice:

- più pulito;
- più estendibile;
- più facile da leggere;
- più facile da modificare;
- più vicino a una struttura reale.

La Versione 3 è quindi un passaggio importante verso un agente più maturo.

Test principali

I test verificano che l'agente continui a funzionare dopo la modifica del parser.

Per esempio:

```
agent("What is 2 + 3?")
```

deve restituire:

```
5
```

Anche:

```
agent("Hello, what is 2 + 3?")
```

deve scegliere correttamente:

```
"addition"
```

E:

```
agent("Hello, my name is luca.")
```

deve restituire:

```
"Hello Luca, nice to meet you!"
```

Questi test confermano che il nuovo parser è più generico, ma non rompe il comportamento precedente.

Limiti della Versione 3

La Versione 3 è più scalabile, ma ha ancora alcuni limiti:

- il Router è ancora a regole;
- il Parser usa ancora regole manuali;
- non c'è ancora un Planner;
- la memoria è ancora una lista;
- non c'è ancora una LLM;
- i tool sono ancora registrati manualmente;
- l'agente esegue ancora una sola azione alla volta.

Questi limiti sono normali e verranno affrontati nelle prossime versioni.

Conclusione

La Versione 3 conclude una prima fase importante del progetto.

Abbiamo ora una struttura con componenti separati:

```
Router  
Generic Parser  
Executor  
Agent  
Memory
```

La novità principale è che il parser non dipende più da condizioni rigide per ogni combinazione di parametri.

Ora legge i parametri richiesti dal tool e usa extractor dedicati per costruire gli argomenti.

Questo rende il progetto più pulito e pronto per evolvere.

Il prossimo passo sarà la Versione 4, dove introdurremo il Planner.

Con il Planner l'agente inizierà a creare un piccolo piano prima di eseguire un'azione.